

Skill Manifest Specification

The skill manifest defines how a **skill** is registered and interpreted by the RAGenius Execution Subsystem. A skill is the primary reusable unit of work; it declares what inputs it accepts, what outputs it returns, which tools it requires, and how its internal workflow should be executed. A formal manifest format makes it possible for the execution engine to load, validate and execute skills deterministically.

Purpose

This document specifies the fields and structure of a skill manifest. It also provides a concrete example and guidance for versioning and validation. The goal is to remove ambiguity so that coding agents and automated registries can reliably parse and validate skills.

Manifest Structure

Each skill is represented as a JSON or YAML object with the following top-level properties:

Field	Type	Description
<code>id</code>	string (unique)	Unique identifier for the skill. Should be stable across versions and free of spaces.
<code>name</code>	string	Human-friendly name for the skill.
<code>version</code>	string	Semantic version (e.g. <code>1.0.0</code>). Increment when the interface or behavior changes.
<code>description</code>	string	Brief description of what the skill does.
<code>input_schema</code>	JSON Schema object	Schema defining the expected input properties and their types. The execution engine must validate incoming input against this schema before running the workflow.
<code>output_schema</code>	JSON Schema object	Schema describing the structure of the final result produced by the skill. This ensures downstream consumers know what to expect.
<code>required_tools</code>	array of strings	List of tool identifiers that the skill depends on. These tools must be registered in the tool registry and available at execution time.
<code>required_permissions</code>	array of strings	List of permission scopes that the skill needs in order to execute its tools. These are checked by the permission engine before any side effect.

Field	Type	Description
<code>workflow_definition</code>	object	Definition of the internal workflow. See the Workflow Runtime Semantics document for the structure of workflows.
<code>enabled</code>	boolean	Whether the skill is active and can be executed. Disabled skills are rejected by the execution subsystem.
<code>created_at</code>	string (timestamp)	When the skill manifest was created (ISO 8601).
<code>updated_at</code>	string (timestamp)	When the skill manifest was last updated (ISO 8601).

Optional Fields

Field	Type	Description
<code>author</code>	string	Name or identifier of the creator of the skill.
<code>tags</code>	array of strings	Optional keywords to categorize the skill (e.g. <code>text</code> , <code>video</code> , <code>report</code>).
<code>dependencies</code>	array of strings	Other skill identifiers that must be executed prior to this skill.
<code>compensation_steps</code>	array	Optional compensation steps to run if the workflow fails after side effects.

Example Manifest (YAML)

```

id: video_director_skill
name: Video Director Skill
version: 1.0.0
description:
Generates a short explainer video using a given prompt and optional context.
input_schema:
  type: object
  required:
    - prompt
    - duration
  properties:
    prompt:
      type: string
      description: The subject or question to cover in the video.
    duration:
      type: number

```

```

    minimum: 1
    maximum: 300
    description: Length of the video in seconds.
  context:
    type: string
    description: Optional extra text to include in the video script.
  output_schema:
    type: object
    required:
      - title
      - summary
    properties:
      title:
        type: string
        description: Title of the generated video.
      summary:
        type: string
        description: Brief description of the video content.
      file_id:
        type: string
        description: Identifier of the video file created by the tool.
  required_tools:
    - rag_retrieval_tool
    - mock_video_generation_tool
  required_permissions:
    - rag.read
    - external_api.write
  workflow_definition:
    steps:
      - id: validate_prompt
        type: validation
        action: ensurePromptLength
      - id: retrieve_context
        type: tool_call
        tool_id: rag_retrieval_tool
        input_mapping:
          query: $.input.prompt
          topK: 3
        output_mapping:
          context: $.output.items
      - id: generate_video
        type: tool_call
        tool_id: mock_video_generation_tool
        input_mapping:
          prompt: $.input.prompt
          duration: $.input.duration
          context: $.steps.retrieve_context.output.context
        output_mapping:

```

```
title: $.output.title
summary: $.output.summary
file_id: $.output.file_id
enabled: true
created_at: 2026-05-09T00:00:00Z
updated_at: 2026-05-09T00:00:00Z
```

Validation

1. **Schema validation:** The execution subsystem should validate incoming requests against the `input_schema` defined in the manifest. Any mismatches should result in a validation error.
2. **Tool availability:** Before executing, the engine must check that all `required_tools` are registered and enabled. Missing tools result in a failure.
3. **Permissions:** The permission engine must ensure the execution context has all `required_permissions`. If a tool requires a higher permission than declared, the skill should be rejected.
4. **Workflow definition:** The workflow should be validated for well-formed steps (unique IDs, valid types, properly mapped inputs/outputs). See the Workflow Runtime Semantics document for validation rules.

Versioning

Use semantic versioning. Increment the major version when you introduce breaking changes (e.g. changing input fields or workflow behavior). Minor version increments indicate backward-compatible enhancements, and patch versions fix minor issues without changing the interface.

Usage Guidelines

- Store skill manifests in a version-controlled repository or database.
 - Provide an admin or CLI interface to register, update, enable or disable skills.
 - Validate the manifest at registration time to avoid runtime surprises.
 - Include documentation strings and examples so that developers know how to call the skill correctly.
 - When deprecating a skill, disable it rather than deleting, so that historical executions can still be traced.
-